

Challenge 4：技能分享与传播（Skill Sharing & Propagation）

🔥 这是整个 Elite20 体系最关键的挑战。本质是：把个人能力 → 可复制的 Agent Skill → 被他人使用 → 自我迭代优化。

一、挑战目标

每个学生提供一个明确、可复用的技能（Skill），教会别人如何使用，让别人真正用起来。

不是写一篇教程，不是分享一个 tip——而是交付一个可安装、可执行、可验证的技能包，附带完整的使用说明和真实案例。

二、技能四条件

你提交的技能必须同时满足以下四个条件：

条件	含义	检验方法
可复用 (Reusable)	别人拿过去能直接用，不依赖你的特定环境	让一个陌生人按你的说明操作，能成功吗？
可执行 (Executable)	不是理论，是能跑的东西	有可运行的 prompt / 代码 / workflow
可验证 (Verifiable)	有明确的输入输出，能判断成功与否	给定输入 X，能得到预期输出 Y 吗？
IO 明确 (Clear I/O)	输入什么、输出什么，一目了然	一句话能说清“输入____，输出____”

三、技能类型（任选方向）

类型	示例

Prompt / AI Skill	高级 prompt 模板、agent workflow、CLAUDE.md / skill.md
技术技能	Claude Code 使用套路、自动化脚本、数据分析 pipeline
生产力技能	用 AI 写论文、做 PPT、做营销
业务技能	招标书生成、用户调研自动化、产品设计流程

你不需要会编程。“用 AI 做高质量 PPT 的完整流程”也是一个技能，只要别人能拿着你的说明复现出来。

四、必须提交的内容

文件	命名格式	说明
Skill 说明文档	姓名_C4_skill说明.md	解决什么问题、使用场景、输入输出、使用步骤、真实案例
可执行内容	姓名_C4_技能名.skill 或 .py 或 .md	可安装的 .skill 包 / prompt / 代码 / workflow
Demo	姓名_C4_demo.mp4 或截图	视频、截图、或 live demo
教学说明	姓名_C4_教学说明.md	如何上手、常见坑、优化技巧——关键是教别人用
⚡ AI 日志	姓名_C4_AI日志.md	用了什么 AI、怎么指挥的、迭代了几轮、你做了什么

五、评判标准

权重	指标	说明
🥇 最高	被使用次数	有多少人实际用过你的技能？是否产生了结果？
🥈 次要	使用效果反馈	别人用了觉得怎么样？
🥉 次要	复用性 / 易用性	拿过去能不能直接用？上手难不难？

🏆 前三名直接录取 Elite20。

注意：不是谁的技能“最厉害”谁赢——而是谁的技能**被最多人用了**。一个简单但人人都用的技能，胜过一个复杂但没人碰的技能。

六、群内传播机制（核心）

提交技能只是开始，真正的价值在传播链：

发布技能到群 → 别人提问 → 你回答/帮助 → 别人成功使用 → 反馈改进 → 你迭代 v2

这个循环跑得越快、覆盖人越多，你的 C4 分数越高。同时也自动为你的 C3（高质量参与）和 C7（内容传播）加分。

七、技能包的结构（.skill 文件标准）

一个 Claude Skill 包的标准结构如下：

```
技能名/
├── SKILL.md           (必须) 主指令文件
│   ├── YAML frontmatter → name + description (触发入口)
│   └── Markdown body    → 工作流程、步骤、代码、示例、边界情况
├── scripts/           (可选) 可执行脚本
│   └── xxx.py          → 确定性/重复性任务用脚本
├── references/         (可选) 参考资料
│   └── xxx.yaml / .md  → 按需加载的文档
└── assets/            (可选) 模板、图标、字体
```

三层加载机制：

1. **元数据** (name + description) — 始终在上下文中，决定是否触发（约100词）
2. **SKILL.md 正文** — 触发时加载（理想 < 500 行）
3. **附属资源** — 按需加载（无限制，脚本可不加载直接执行）

打包后的 `.skill` 文件就是一个 tar.gz 压缩包，可以直接安装到 Claude 环境中。

八、两个示例技能详解

以下是本次对话中实际创建的两个真实技能，作为 C4 提交的参考样板。两个 `.skill` 文件已附在本文档同一群消息中，可直接下载安装使用。

示例 1: wechat-doc-mapper（微信文档挑战映射器）

一句话说明

输入一个微信同步文件夹路径，输出按作者×挑战分组的文档清单表 + Excel 文件 + 缺口分析。

解决什么问题

Elite20 群里大家往群里扔文件，但没人知道：谁交了什么？哪个挑战还缺什么？哪些文件没有按规范命名？手动整理费时费力，而且容易漏。

这个技能自动化了整个盘点流程：扫描 → 识别作者 → 识别格式和标题 → 推断用途 → 映射到挑战 → 生成报告。

技能架构

```
wechat-doc-mapper/
├── SKILL.md (主指令, 322 行)
│   ├── 前置条件: 命名规范 + 回退提取链
│   ├── 6 步工作流: 路径 → 作者识别 → 盘点 → 标题提取 → 挑战映射 → 输出
│   └── 边界情况: 空文件夹、超大文件、编码问题、命名不规范等
├── scripts/
│   └── wechat_doc_mapper.py (19.4 KB - 扫描器 + 映射器 + Excel 生成器)
└── references/
    └── challenges.yaml (C1-C7 挑战注册表, 含信号关键词)
```

核心设计

- 作者优先：**先识别谁提交的（从文件名规范 → PDF/DOCX 元数据 → 文件夹名 → 文档开头），再分析内容。就像收作业——先看名字，再看内容。
- 信号评分映射：**每个挑战在 YAML 中定义了关键词（如 C4 的 `skill`、`agent`、`workflow`）、扩展名（如 `.skill`）、路径片段（如 `/demo/`）。脚本对每个文件计算多维信号分数，自动分配主挑战和副挑战。
- 缺口分析：**对比每个作者的提交和挑战的“预期产出物清单”，自动报告缺了什么。例如：“LiMing 的 C2 缺少 `references.bib` 和 `summary.md`”。
- 双格式输出：**Markdown 表格（在聊天中直接显示）+ 4 页 Excel（文档清单、作者×挑战矩阵、缺口分析、挑战参考）。

使用方式

安装 `.skill` 文件后，对 Claude 说：

“扫描 `~/Documents/WeChat Files/Elite20群`，把所有文件映射到挑战上”

或英文：

“Scan this folder and map documents to challenges: `/path/to/folder`”

真实测试结果

用 9 个模拟文件测试：

挑战	文件数	识别结果
C1 课程资料	4	syllabus（原文+译文）、slides、method.md 
C2 论文	2	paper.tex、references.bib 
C3 参与	2	interaction_log.md、reflection.md 
C0 未分类	1	misc_notes.txt 

缺口分析正确检测到 C2 缺少 `paper.pdf` 和 `summary.md`。

可扩展性

挑战注册表是 YAML 驱动的。要新增挑战，只需在 `challenges.yaml` 里追加条目，不需要改任何代码。当前已包含完整的 C1–C7 定义。

示例 2：skill-explainer（技能 X 光分析器）

一句话说明

输入任意一个 `.skill` 文件或技能目录，输出一份结构化的 Markdown 分析报告，涵盖用途、架构、用法、评审和改进建议。

解决什么问题

当你拿到一个别人写的 skill，你怎么知道它是干什么的？写得好不好？能不能用？有没有坑？

手动读 SKILL.md 可以，但你可能漏掉：描述质量差导致不会被触发、缺少边界情况处理、目录

结构不规范、假设了不存在的前置知识.....

这个元技能像一个“技能评审员”：系统性地拆解技能的每一层，用一套标准化的评审维度（D1-D4）打分，并给出具体可操作的改进建议。

技能架构

```
skill-explainer/
├── SKILL.md                      (主指令, 231 行)
│   ├── 4 步 workflow: 解析 → 扫描目录 → 分析 → 生成报告
│   ├── 7 节报告模板 (身份、用途、架构、参考、用法、评审、建议)
│   └── 边界情况: 空 SKILL.md、无 frontmatter、超长文件、损坏的 .skill 包等
├── references/
│   └── rubric.md                 (评审维度标准 D1-D4, 含评分表和常见陷阱)
```

核心设计

- 1. **纯指令技能**：没有脚本——它教 LLM 如何分析，而不是用代码分析。这本身就展示了技能的两种形态：指令驱动 vs 脚本驱动。
- 2. **四维评审矩阵**（来自 `references/rubric.md`）：

维度	权重	评什么
D1 描述质量	高	触发短语够不够多？中英文都有吗？够“主动”吗？
D2 结构完整性	高	有没有清晰的工作流？有没有示例？有没有边界情况？
D3 命名与组织	中	kebab-case？标准目录结构？文件名自解释？
D4 清晰度与可教性	中	一个全新的 Claude 实例能无歧义地执行吗？

每个维度用  /  /  三级评分，每个评审点配套具体改进建议。

- 3. **报告模板固定**：7 个 section 必须全部输出，确保每次分析的一致性和完整性。从“这是什么” → “怎么用” → “好不好” → “怎么改”——逻辑递进。

使用方式

安装 `.skill` 文件后，对 Claude 说：

```
“帮我分析一下这个技能：/path/to/some-skill”
```

或：

“Skill X-ray on wechat-doc-mapper.skill”

或上传一个 `.skill` 文件后说：“这个技能怎么样？”

真实测试结果

测试对象：wechat-doc-mapper

维度	评级	核心发现
D1 描述质量	✅ 良好	9 个触发短语（4中文+5英文），描述了“做什么”和“何时用”
D2 结构完整性	✅ 良好	6 步清晰流水线，含代码块和边界情况
D3 命名与组织	✅ 良好	kebab-case，标准 scripts/ + references/ 结构
D4 清晰度与可教性	⚠️ 需改进	默认读者了解 Elite20 挑战含义，对外部用户不够自解释

生成了 5 条排序建议，最高优先级：增加一段背景说明 Elite20 是什么。

自测试（分析自己）： 正确识别自身为纯指令技能（无脚本，不扣分），13 个触发短语，231 行（在 500 行限制内）。

九、如何从零开始创建你的第一个技能

Step 1: 找到你的“可复制能力”

问自己：我经常重复做的、别人也需要的事情是什么？

举例：

- 我每次写 PPT 都用同一套 AI workflow → 可以变成技能
- 我有一个很好用的翻译 prompt 模板 → 可以变成技能
- 我会用 Claude Code 自动处理 Excel → 可以变成技能
- 我有一套面试准备流程 → 可以变成技能

Step 2: 写 SKILL.md

按标准结构写：

```
---
name: 你的技能名 (kebab-case)
```

```
description: >
    一段话描述技能做什么 + 何时触发。列出 3-5 个触发短语。
    越具体越好，越"主动"越好。
---

# 技能标题

## Purpose (为什么需要这个技能)

## Workflow (怎么用，分步骤)

### Step 1 - ...
### Step 2 - ...

## Edge Cases (边界情况)

## Examples (使用示例)
```

Step 3：测试

自己用一遍。找一个同学用一遍。记录他们遇到的问题。

Step 4：发到群里

按命名规范上传：

```
YourName_C4_skill1说明.md
YourName_C4_技能名.skill1 (或 .md / .py)
YourName_C4_demo截图.png
YourName_C4_教学说明.md
YourName_C4_AI日志.md
```

附一句话：“这是我的 C4 技能——XXX，输入____就能得到____，欢迎大家试用和反馈！”

Step 5：迭代

根据群友反馈改进，提交 _v2 、 _v3 。每一次迭代都会被记录为“进步”。

十、附件

本文档随附两个真实的 .skill 文件，可直接安装使用：

文件	说明	安装后触发方式
----	----	---------

wechat-doc-mapper.skill	微信文档挑战映射器 (脚本驱动型)	"扫描微信文件夹" / "文档分类" / "map documents to challenges"
skill-explainer.skill	技能 X 光分析器 (指令驱动型)	"分析这个技能" / "skill X-ray" / "技能解释"

这两个技能分别展示了技能的两种形态：

- **wechat-doc-mapper** = 指令 + 脚本 + 数据 → 适合需要确定性计算的任务
- **skill-explainer** = 纯指令 + 评审标准 → 适合需要判断力和分析力的任务

两种都是有效的技能形态，选择取决于你的任务性质。

📢 **C4 的本质价值：从“学习” → “能力产品化”。** 一个技能被 10 人用 = 放大 10 倍价值，被 100 人用 = 放大 100 倍。完全符合 NEOLAF 三阶段：Before（理解技能）→ During（用技能做事）→ After（分享 + 优化 + 传播）。